



Workers API reference

The in-Worker R2 API is accessed by binding an R2 bucket to a [Worker](#). The Worker you write can expose external access to buckets via a route or manipulate R2 objects internally.

The R2 API includes some extensions and semantic differences from the S3 API. If you need S3 compatibility, consider using the [S3-compatible API](#).

Concepts

R2 organizes the data you store, called objects, into containers, called buckets. Buckets are the fundamental unit of performance, scaling, and access within R2.

Create a binding

Bindings

A binding is how your Worker interacts with external resources such as [KV Namespaces](#), [Durable Objects](#), or [R2 Buckets](#). A binding is a runtime variable that the Workers runtime provides to your code. You can declare a variable name in your Wrangler file that will be bound to these resources at runtime, and interact with them through this variable. Every binding's variable name and behavior is determined by you when deploying the Worker. Refer to [Environment Variables](#) for more information.

A binding is defined in the Wrangler file of your Worker project's directory.

To bind your R2 bucket to your Worker, add the following to your Wrangler file. Update the binding property to a valid JavaScript variable identifier and `bucket_name` to the name of your R2 bucket:

`{}` `wrangler.jsonc`

 `wrangler.toml`

```
{
  "r2_buckets": [
    {
      "binding": "MY_BUCKET",
      "bucket_name": "<YOUR_BUCKET_NAME>"
    }
  ]
}
```

Within your Worker, your bucket binding is now available under the `MY_BUCKET` variable and you can begin interacting with it using the [bucket methods](#) described below.

Bucket method definitions

The following methods are available on the bucket binding object injected into your code.

For example, to issue a PUT object request using the binding above:

JavaScript

```
export default {
  async fetch(request, env) {
    const url = new URL(request.url);
    const key = url.pathname.slice(1);

    switch (request.method) {
      case "PUT":
        await env.MY_BUCKET.put(key, request.body);
        return new Response(`Put ${key} successfully!`);

      default:
        return new Response(`${request.method} is not allowed`, {
          status: 405,
          headers: {
            Allow: "PUT",
          },
        });
    }
  },
};
```

- **head** (key: string): Promise<R2Object | null>
 - Retrieves the R2Object for the given key containing only object metadata, if the key exists, and null if the key does not exist.
- **get** (key: string, options?: R2GetOptions): Promise<R2ObjectBody | R2Object | null>
 - Retrieves the R2ObjectBody for the given key containing object metadata and the object body as a ReadableStream, if the key exists, and null if the key does not exist.
 - In the event that a precondition specified in options fails, get() returns an R2Object with body undefined.
- **put**

(key: string, value: ReadableStream | ArrayBuffer | ArrayBufferView | string | null | Blob, options?: R2PutOptions)

- Stores the given value and metadata under the associated key. Once the write succeeds, returns an `R2Object` containing metadata about the stored Object.
- In the event that a precondition specified in `options` fails, `put()` returns `null`, and the object will not be stored.
- R2 writes are strongly consistent. Once the Promise resolves, all subsequent read operations will see this key value pair globally.
- **delete** `(key: string | string[]): Promise<void>`
 - Deletes the given values and metadata under the associated keys. Once the delete succeeds, returns `void`.
 - R2 deletes are strongly consistent. Once the Promise resolves, all subsequent read operations will no longer see the provided key value pairs globally.
 - Up to 1000 keys may be deleted per call.
- **list** `(options?: R2ListOptions): Promise<R2Objects>`
 - Returns an `R2Objects` containing a list of `R2Object` contained within the bucket.
 - The returned list of objects is ordered lexicographically.
 - Returns up to 1000 entries, but may return less in order to minimize memory pressure within the Worker.
 - To explicitly set the number of objects to list, provide an [R2ListOptions](#) object with the `limit` property set.
- **createMultipartUpload**
`(key: string, options?: R2MultipartOptions): Promise<R2MultipartUpload>`
 - Creates a multipart upload.
 - Returns Promise which resolves to an `R2MultipartUpload` object representing the newly created multipart upload. Once the multipart upload has been created, the multipart upload can be immediately

interacted with globally, either through the Workers API, or through the S3 API.

- `resumeMultipartUpload` **(key: string, uploadId: string): R2MultipartUpload**
 - Returns an object representing a multipart upload with the given key and uploadId.
 - The `resumeMultipartUpload` operation does not perform any checks to ensure the validity of the uploadId, nor does it verify the existence of a corresponding active multipart upload. This is done to minimize latency before being able to call subsequent operations on the `R2MultipartUpload` object.

R2object definition

`R2object` is created when you PUT an object into an R2 bucket. `R2object` represents the metadata of an object based on the information provided by the uploader. Every object that you PUT into an R2 bucket will have an `R2object` created.

- `key` **string**
 - The object's key.
- `version` **string**
 - Random unique string associated with a specific upload of a key.
- `size` **number**
 - Size of the object in bytes.
- `etag` **string**

Note

Cloudflare recommends using the `httpEtag` field when returning an etag in a response header. This ensures the etag is quoted and conforms to [RFC 9110](#).

- The etag associated with the object upload.
- `httpEtag` **string**
 - The object's etag, in quotes so as to be returned as a header.
- `uploaded` **Date**
 - A Date object representing the time the object was uploaded.
- `httpMetadata` **R2HTTPMetadata**
 - Various HTTP headers associated with the object. Refer to [HTTP Metadata](#).
- `customMetadata` **Record<string, string>**
 - A map of custom, user-defined metadata associated with the object.
- `range` **R2Range**
 - A R2Range object containing the returned range of the object.
- `checksums` **R2Checksums**
 - A R2Checksums object containing the stored checksums of the object. Refer to [checksums](#).
- `writeHttpMetadata` **(headers: Headers): void**
 - Retrieves the `httpMetadata` from the `R2Object` and applies their corresponding HTTP headers to the Headers input object. Refer to [HTTP Metadata](#).
- `storageClass` **'Standard' | 'InfrequentAccess'**
 - The storage class associated with the object. Refer to [Storage Classes](#).

- `ssecKeyMd5` `string`
 - Hex-encoded MD5 hash of the [SSE-C](#) key used for encryption (if one was provided). Hash can be used to identify which key is needed to decrypt object.

R2objectBody definition

`R2objectBody` represents an object's metadata combined with its body. It is returned when you GET an object from an R2 bucket. The full list of keys for `R2objectBody` includes the list below and all keys inherited from [R2Object](#).

- `body` `ReadableStream`
 - The object's value.
- `bodyUsed` `boolean`
 - Whether the object's value has been consumed or not.
- `arrayBuffer` `() : Promise<ArrayBuffer>`
 - Returns a Promise that resolves to an `ArrayBuffer` containing the object's value.
- `text` `() : Promise<string>`
 - Returns a Promise that resolves to a string containing the object's value.
- `json` `<T>() : Promise<T>`
 - Returns a Promise that resolves to the given object containing the object's value.
- `blob` `() : Promise<Blob>`
 - Returns a Promise that resolves to a binary `Blob` containing the object's value.

R2MultipartUpload definition

An R2MultipartUpload object is created when you call `createMultipartUpload` or `resumeMultipartUpload`. R2MultipartUpload is a representation of an ongoing multipart upload.

Uncompleted multipart uploads will be automatically aborted after 7 days.

Note

An R2MultipartUpload object does not guarantee that there is an active underlying multipart upload corresponding to that object.

A multipart upload can be completed or aborted at any time, either through the S3 API, or by a parallel invocation of your Worker. Therefore it is important to add the necessary error handling code around each operation on a R2MultipartUpload object in case the underlying multipart upload no longer exists.

- `key` `string`
 - The key for the multipart upload.
- `uploadId` `string`
 - The uploadId for the multipart upload.
- `uploadPart`
(`partNumber`: `number`, `value`: `ReadableStream` | `ArrayBuffer` | `ArrayBufferView` | `string` | `Blob`, `options`?: `R2MultipartUploadOptions`)
 - Uploads a single part with the specified part number to this multipart upload. Each part must be uniform in size with an exception for the final part which can be smaller.
 - Returns an `R2UploadedPart` object containing the `etag` and `partNumber`. These `R2UploadedPart` objects are required when completing the multipart upload.
- `abort` `()`: `Promise<void>`

- Aborts the multipart upload. Returns a Promise that resolves when the upload has been successfully aborted.
- **complete** `(uploadedParts: R2UploadedPart[]): Promise<R2Object>`
 - Completes the multipart upload with the given parts.
 - Returns a Promise that resolves when the complete operation has finished. Once this happens, the object is immediately accessible globally by any subsequent read operation.

Method-specific types

R2GetOptions

- **onlyIf** `R2Conditional | Headers`
 - Specifies that the object should only be returned given satisfaction of certain conditions in the `R2Conditional` or in the conditional Headers. Refer to [Conditional operations](#).
- **range** `R2Range`
 - Specifies that only a specific length (from an optional offset) or suffix of bytes from the object should be returned. Refer to [Ranged reads](#).
- **sseKey** `ArrayBuffer | string`
 - Specifies a key to be used for [SSE-C](#). Key must be 32 bytes in length, in the form of a hex-encoded string or an `ArrayBuffer`.

Ranged reads

`R2GetOptions` accepts a `range` parameter, which can be used to restrict the data returned in body.

There are 3 variations of arguments that can be used in a range:

- An offset with an optional length.

- An optional offset with a length.
- A suffix.
- `offset` `number`
 - The byte to begin returning data from, inclusive.
- `length` `number`
 - The number of bytes to return. If more bytes are requested than exist in the object, fewer bytes than this number may be returned.
- `suffix` `number`
 - The number of bytes to return from the end of the file, starting from the last byte. If more bytes are requested than exist in the object, fewer bytes than this number may be returned.

R2PutOptions

- `onlyIf` `R2Conditional | Headers`
 - Specifies that the object should only be stored given satisfaction of certain conditions in the `R2Conditional`. Refer to [Conditional operations](#).
- `httpMetadata` `R2HTTPMetadata | Headers` optional
 - Various HTTP headers associated with the object. Refer to [HTTP Metadata](#).
- `customMetadata` `Record<string, string>` optional
 - A map of custom, user-defined metadata that will be stored with the object.

Note

Only a single hashing algorithm can be specified at once.

- `md5` `ArrayBuffer | string` optional

- A md5 hash to use to check the received object's integrity.
- sha1 **ArrayBuffer | string** optional
 - A SHA-1 hash to use to check the received object's integrity.
- sha256 **ArrayBuffer | string** optional
 - A SHA-256 hash to use to check the received object's integrity.
- sha384 **ArrayBuffer | string** optional
 - A SHA-384 hash to use to check the received object's integrity.
- sha512 **ArrayBuffer | string** optional
 - A SHA-512 hash to use to check the received object's integrity.
- storageClass **'Standard' | 'InfrequentAccess'**
 - Sets the storage class of the object if provided. Otherwise, the object will be stored in the default storage class associated with the bucket. Refer to [Storage Classes](#).
- ssekKey **ArrayBuffer | string**
 - Specifies a key to be used for [SSE-C](#). Key must be 32 bytes in length, in the form of a hex-encoded string or an ArrayBuffer.

R2MultipartOptions

- httpMetadata **R2HTTPMetadata | Headers** optional
 - Various HTTP headers associated with the object. Refer to [HTTP Metadata](#).
- customMetadata **Record<string, string>** optional
 - A map of custom, user-defined metadata that will be stored with the object.
- storageClass **string**

- Sets the storage class of the object if provided. Otherwise, the object will be stored in the default storage class associated with the bucket. Refer to [Storage Classes](#).
- `sseKey` `ArrayBuffer | string`
 - Specifies a key to be used for [SSE-C](#). Key must be 32 bytes in length, in the form of a hex-encoded string or an `ArrayBuffer`.

R2ListOptions

- `limit` `number` optional
 - The number of results to return. Defaults to 1000, with a maximum of 1000.
 - If `include` is set, you may receive fewer than `limit` results in your response to accommodate metadata.
- `prefix` `string` optional
 - The prefix to match keys against. Keys will only be returned if they start with given prefix.
- `cursor` `string` optional
 - An opaque token that indicates where to continue listing objects from. A cursor can be retrieved from a previous list operation.
- `delimiter` `string` optional
 - The character to use when grouping keys.
- `include` `Array<string>` optional
 - Can include `httpMetadata` and/or `customMetadata`. If included, items returned by the list will include the specified metadata.

- Note that there is a limit on the total amount of data that a single `list` operation can return. If you request data, you may receive fewer than `limit` results in your response to accommodate metadata.
- The [compatibility_date](#) must be set to 2022-08-04 or later in your Wrangler file. If not, then the `r2_list_honor_include` compatibility flag must be set. Otherwise it is treated as `include: [ 'httpMetadata', 'customMetadata' ]` regardless of what the `include` option provided actually is.

This means applications must be careful to avoid comparing the amount of returned objects against your `limit`. Instead, use the `truncated` property to determine if the `list` request has more data to be returned.

JavaScript

```
const options = {
  limit: 500,
  include: ["customMetadata"],
};

const listed = await env.MY_BUCKET.list(options);

let truncated = listed.truncated;
let cursor = truncated ? listed.cursor : undefined;

// ❌ - if your limit can't fit into a single response or your
// bucket has less objects than the limit, it will get stuck here.
while (listed.objects.length < options.limit) {
  // ...
}

// ✅ - use the truncated property to check if there are more
// objects to be returned
while (truncated) {
  const next = await env.MY_BUCKET.list({
    ...options,
    cursor: cursor,
  });
  listed.objects.push(...next.objects);

  truncated = next.truncated;
  cursor = next.cursor;
}
```

R2Objects

An object containing an R2Object array, returned by `BUCKET_BINDING.list()`.

- `objects` `Array<R2Object>`

- An array of objects matching the `list` request.
- `truncated` `boolean`
 - If `true`, indicates there are more results to be retrieved for the current `list` request.
- `cursor` `string` optional
 - A token that can be passed to future `list` calls to resume listing from that point. Only present if `truncated` is `true`.
- `delimitedPrefixes` `Array<string>`
 - If a delimiter has been specified, contains all prefixes between the specified prefix and the next occurrence of the delimiter.
 - For example, if no prefix is provided and the delimiter is `'/'`, `foo/bar/baz` would return `foo` as a delimited prefix. If `foo/` was passed as a prefix with the same structure and delimiter, `foo/bar` would be returned as a delimited prefix.

Conditional operations

You can pass an `R2Conditional` object to `R2GetOptions` and `R2PutOptions`. If the condition check for `get()` fails, the body will not be returned. This will make `get()` have lower latency.

If the condition check for `put()` fails, `null` will be returned instead of the `R2Object`.

- `etagMatches` `string` optional
 - Performs the operation if the object's etag matches the given string.
- `etagDoesNotMatch` `string` optional
 - Performs the operation if the object's etag does not match the given string.
- `uploadedBefore` `Date` optional

- Performs the operation if the object was uploaded before the given date.
- `uploadedAfter` **Date** optional
 - Performs the operation if the object was uploaded after the given date.

Alternatively, you can pass a `Headers` object containing conditional headers to `R2GetOptions` and `R2PutOptions`. For information on these conditional headers, refer to [the MDN docs on conditional requests](https://developer.mozilla.org/en-US/docs/http/conditional_requests) [↗]. All conditional headers aside from `If-Range` are supported.

For more specific information about conditional requests, refer to [RFC 7232](https://tools.ietf.org/html/rfc7232) [↗].

HTTP Metadata

Generally, these fields match the HTTP metadata passed when the object was created. They can be overridden when issuing GET requests, in which case, the given values will be echoed back in the response.

- `contentType` **string** optional
- `contentLanguage` **string** optional
- `contentDisposition` **string** optional
- `contentEncoding` **string** optional
- `cacheControl` **string** optional
- `cacheExpiry` **Date** optional

Checksums

If a checksum was provided when using the `put()` binding, it will be available on the returned object under the `checksums` property. The MD5 checksum will be included by default for non-multipart objects.

- `md5` **ArrayBuffer** optional
 - The MD5 checksum of the object.
- `sha1` **ArrayBuffer** optional
 - The SHA-1 checksum of the object.
- `sha256` **ArrayBuffer** optional
 - The SHA-256 checksum of the object.
- `sha384` **ArrayBuffer** optional
 - The SHA-384 checksum of the object.
- `sha512` **ArrayBuffer** optional
 - The SHA-512 checksum of the object.

R2UploadedPart

An `R2UploadedPart` object represents a part that has been uploaded.

`R2UploadedPart` objects are returned from `uploadPart` operations and must be passed to `completeMultipartUpload` operations.

- `partNumber` **number**
 - The number of the part.
- `etag` **string**
 - The etag of the part.

Storage Class

The storage class where an `R2Object` is stored. The available storage classes are `Standard` and `InfrequentAccess`. Refer to [Storage classes](#) for more information.

Was this helpful?

[Previous](#)[← Use the R2 multipart API from Workers](#)[Next](#)[R2 Data Catalog →](#)

Last updated: Jan 29, 2025

Resources

[API](#)[New to Cloudflare?](#)[Directory](#)[Sponsorships](#)[Open Source](#)

Support

[Help Center](#)[System Status](#)[Compliance](#)[GDPR](#)

Company

[cloudflare.com](#)[Our team](#)[Careers](#)

Tools

[Cloudflare Radar](#)[Speed Test](#)[Is BGP Safe Yet?](#)[RPKI Toolkit](#)[Certificate Transparency](#)

Community

[X](#)[Discord](#)[YouTube](#)[GitHub](#)

